

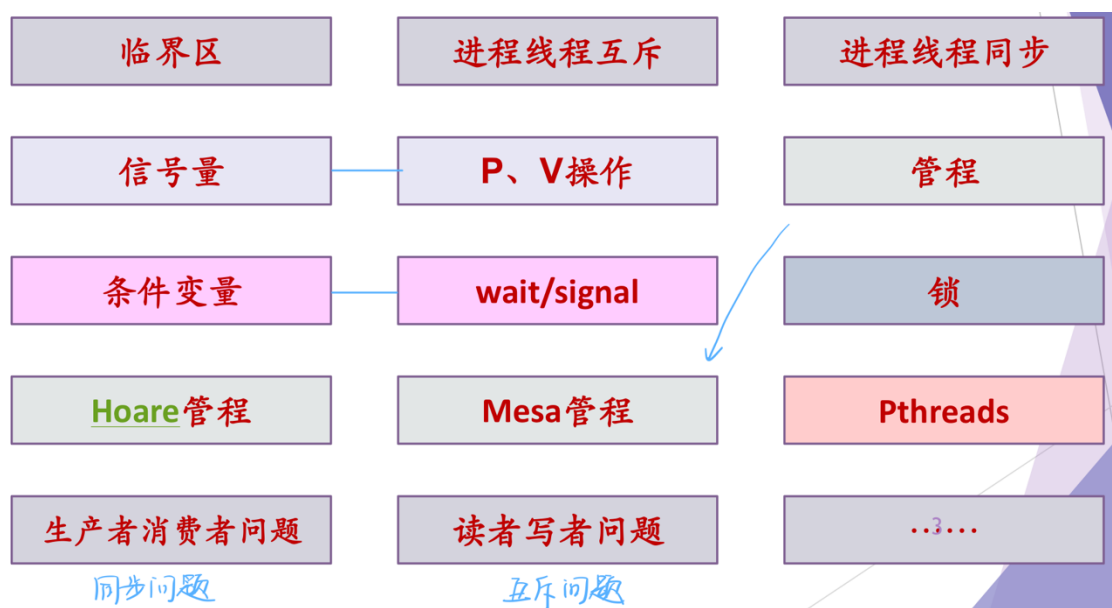
Lecture10 并发机制

问题与概念

1. 问题

- 1) 并发环境下程序执行是否会受到进程/线程调度的影响？会
- 2) 举例说明进程/线程互斥、进程/线程同步的场景及相关问题
- 3) 信号量与 PV 操作是否可以既解决同步（多线程协作）问题又解决互斥（共享资源）问题？
- 4) 锁的实现有那些方法？解决互斥问题
- 5) 条件变量及相应的操作解决的是什么问题？同步问题
- 6) 管程机制主要实现特点是什么？
- 7) Pthread（用户态线程包）解决同步互斥对应的 API 有哪些？

2. 概念



进程/线程的并发执行

1. 进程线程的特征

1) 并发

- a) 执行过程不是连续的，是间断性的
- b) 相对执行速度不可预测

并发是所有问题的基础

并发是操作系统设计的基础

2) 共享有限资源

- a) 进程/线程之间的制约性

3) 不确定性

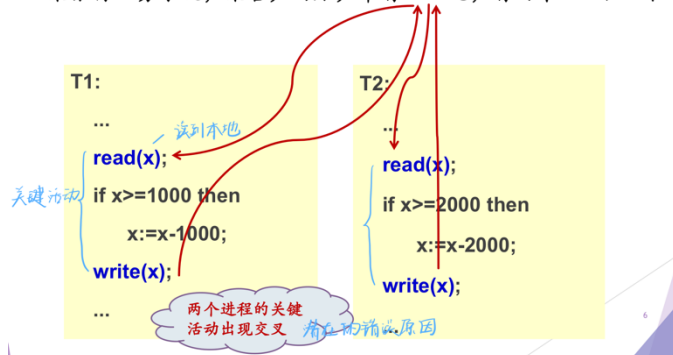
- a) 执行的结果与其执行的相对速度有关，是不确定的

2. 并发环境下进程/线程执行时的问题

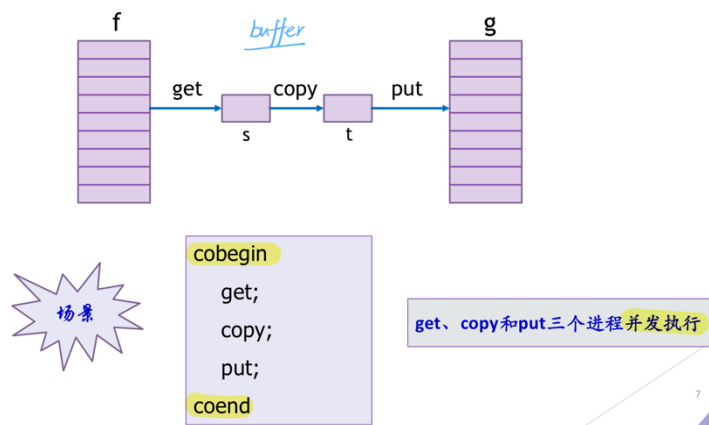
1) 与时间有关的错误

a) 例 1

某银行业务系统，某客户的账户中有5000元，有两个ATM机T1和T2



b) 例 2



- 并发执行过程分析

当前状态 f s t g
 (3,4,...,m) 2 2 (1,2)

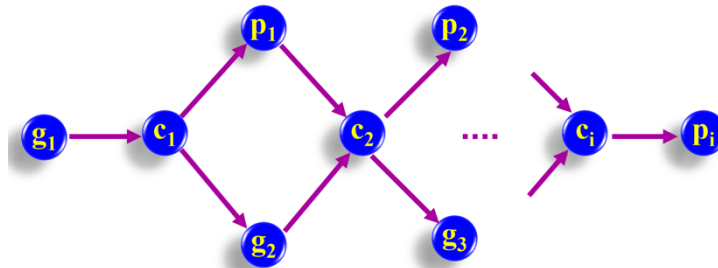
可能的执行 (假设g,c,p为get,copy,put的一次循环)

g,c,p	(4,5,...,m)	3	3	(1,2,3) ✓
g,p,c	(4,5,...,m)	3	3	(1,2,2) ✗
c,g,p	(4,5,...,m)	3	2	(1,2,2) ✗
c,p,g	(4,5,...,m)	3	2	(1,2,2) ✗
p,c,g	(4,5,...,m)	3	2	(1,2,2) ✗
p,g,c	(4,5,...,m)	3	3	(1,2,2) ✗

设信息长度为m, 有多少种可能性? $2 \times 2 \times \dots \times 2 = 2^m$

✓ 前趋图

用于分析并发环境下程序间的制约关系

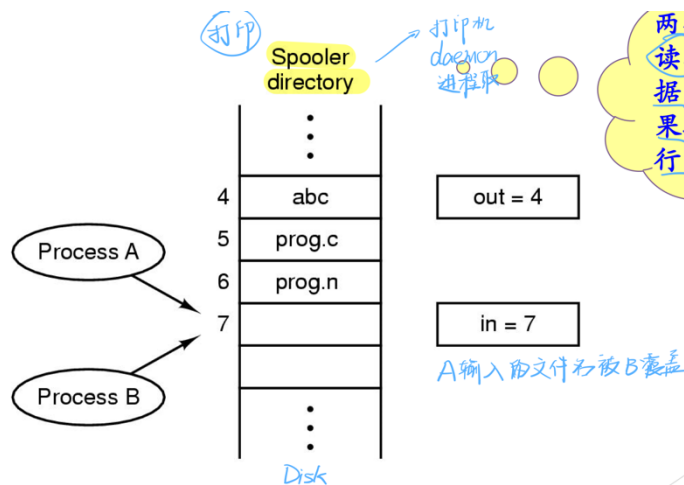


2) 竞争条件 (race condition, 竞态)

a) 简介

两个或多个进程读写某些共享数据, 而最后的结果取决于进程运行的精确时序

b) 例子



看课件上有关 ics 的内容

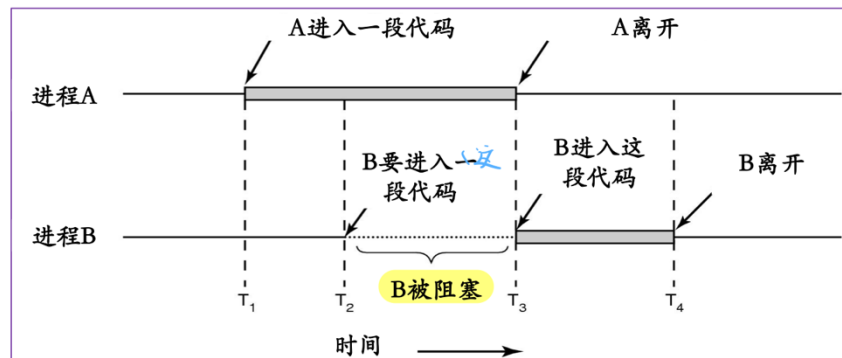
互斥问题与同步问题

1. 进程/线程互斥

1) 概念

a) 互斥

由于各进程要求使用共享资源（变量、文件等），而这些资源需要排他性使用，即各进程之间竞争使用这些资源



b) 临界资源：critical resource

系统中某些资源一次只允许一个进程/线程使用，称这样的资源为临界资源或互斥资源或共享变量

c) 临界区(互斥区)：critical section(region)

各个进程/线程中对某个临界资源（共享变量）实施操作的程序片段（这些片段一般分散在多个进程中）

2) 临界区(互斥区)的使用原则

a) 前提

- 任何进程无权停止其它进程的运行
- 进程之间相对运行速度无硬性规定

b) 必须满足的原则

- 有空让进：当没有进程在临界区时，任何有权使用互斥区的进程可进入临界区
- 无空等待：不允许两个以上的进程同时进入临界区
- 有限等待：任何进入临界区的要求应在有限时间内得到满足

c) 不必满足的原则

- 多中择一：当没有进程在临界区，而同时有多个进程要求进入临

界区时，只能让其中之一进入，其他进程必须等待

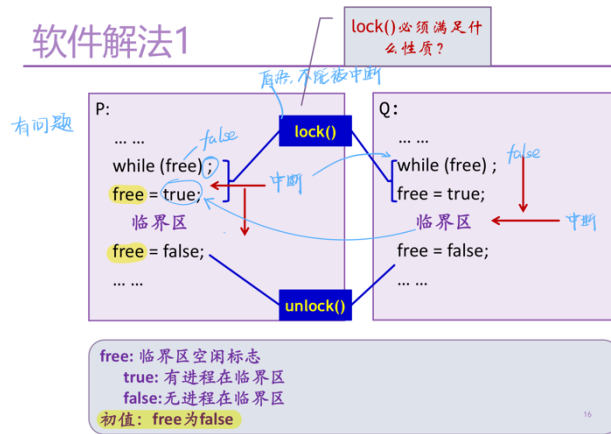
- 让权等待：处于等待状态的进程应放弃占用 CPU，以使其他进程有机会得到 CPU 的使用权

3) 实现进程线程互斥的方案

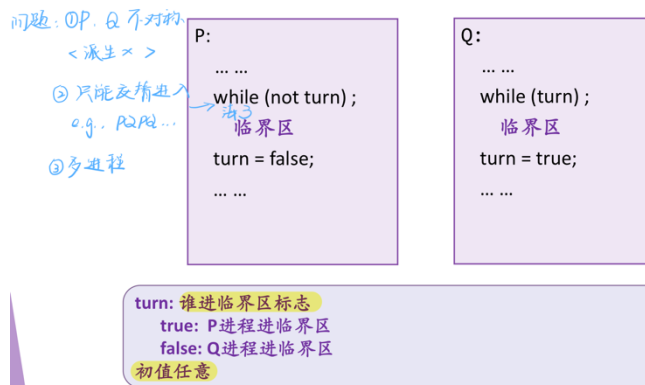
a) 软件方案

- 有问题的解法

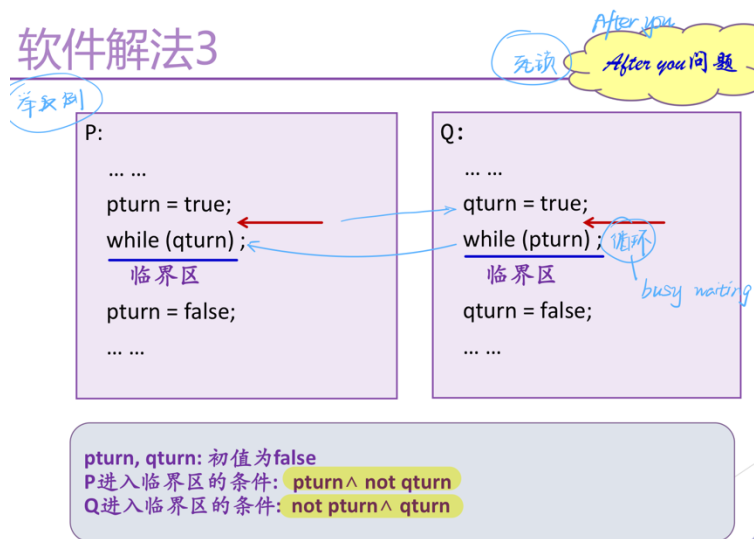
软件解法1



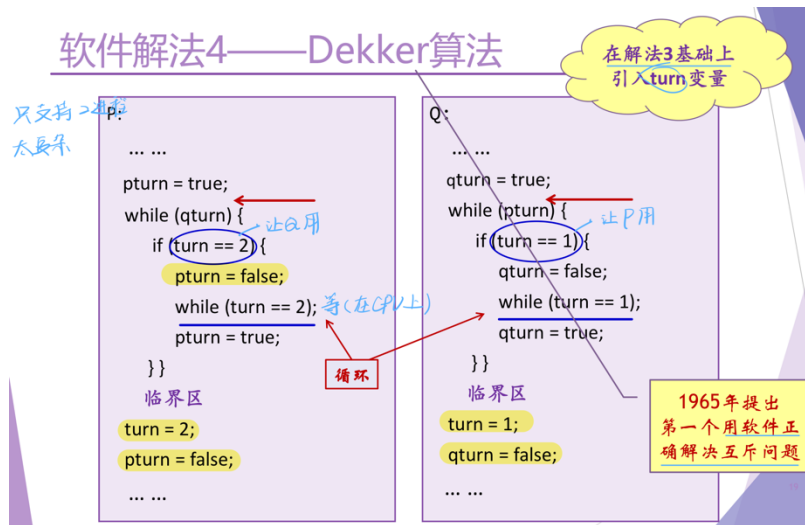
软件解法2



软件解法3

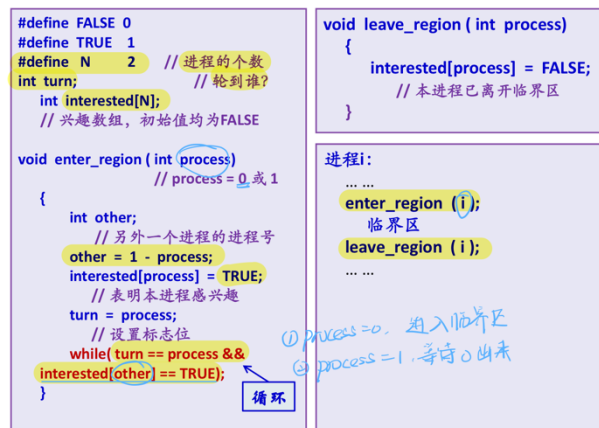


- Dekker 解法



- Peterson 解法

软件解法5——Peterson算法



b) 硬件方案

- 屏蔽中断

硬件解法1——关闭中断方法

“开关中断”指令

```

while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}

```

内核

提问：单处理器下，关闭中断是否符合临界区的使用原则（有空让进，无空等待、有限等待）？

- 简单，高效
 - 代价高，限制CPU并发能力（临界区大小）
 - 不适用于多处理器
 - 适用于操作系统本身，不适用于用户进程
- 不脱大小*

- TSL(XCHG)指令

TSL指令: Test and Set Lock

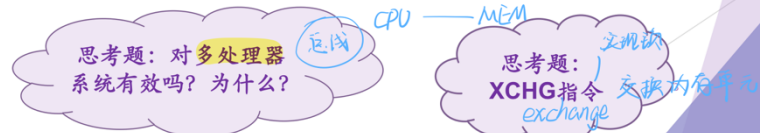
```

enter_region:
    TSL REGISTER, LOCK
    CMP REGISTER, #0
    JNE enter_region
    RET

leave_region:
    MOVE LOCK, #0
    RET
    
```

复制锁到寄存器并将锁置1
判断寄存器内容是否是零?
若不是零, 跳转到enter_region
返回调用者, 进入了临界区

在锁中置0
返回调用者



4) 互斥问题小结

a) 软件方法的问题

需要编程技巧

b) 硬件方法的问题

- 忙等待(busy waiting) (没有实现让权等待, 单处理器上虽然不会导致错误但浪费资源)
 - ✓ 进程在得到临界区访问权之前, 持续测试而不做其他事情
 - ✓ 自旋锁 Spin lock (使多处理器也可用该方法)
- 优先级反转 (倒置)

2. 进程/线程同步 (synchronization)

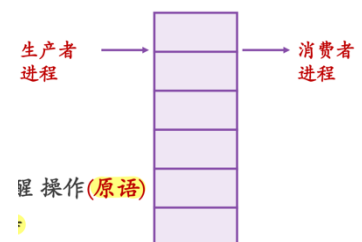
1) 概念

- 指系统中多个进程线程中发生的事件存在某种时序关系, 需要相互合作, 共同完成一项任务
- 具体地说, 一个进程运行到某一点时, 要求另一伙伴进程为它提供消息, 在未获得消息之前, 该进程进入阻塞态, 获得消息后被唤醒进入就绪态

2) 生产者消费者 (有界缓冲区) 问题

a) 问题描述

- 一个或多个生产者生产某种类型的数据放置在缓冲区中
- 有消费者从缓冲区中取数据, 每次取一项



- 只能有一个生产者或消费者对缓冲区进行操作
- b) 要解决的问题
- 当缓冲区已满时，生产者不会继续向其中添加数据
- 当缓冲区为空时，消费者不会从中移走数据
- c) 解决思路
- 提供：睡眠与唤醒操作(原语)
 - 要求：避免忙等待
- d) 解决方案
- 有问题的方案

生产者/消费者问题

```
#define N 100
int count=0;

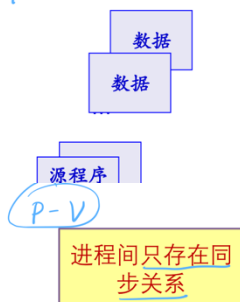
void producer(void)
{ int item;
  while(TRUE) {
    item=produce_item();
    if(count==N) sleep();
    insert_item(item);
    count=count+1;
    if(count==1) wakeup(consumer);
  }
}

void consumer(void)
{
  int item;
  while(TRUE) {
    if(count==0) sleep();
    item=remove_item();
    count=count-1;
    if(count==N-1) wakeup(producer);
    consume_item(item);
  }
}
```

问题
一种场景：消费者判断count=0后进入睡眠前被切换

3) 其他例子

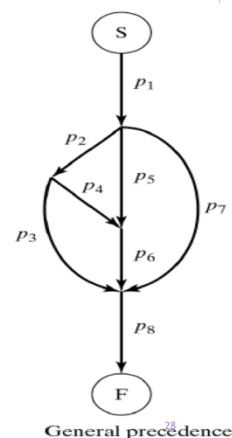
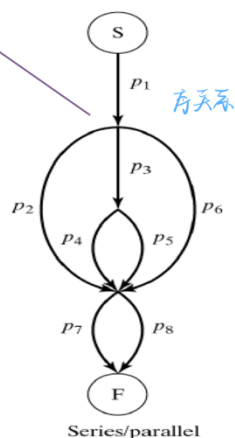
producer - consumer



批处理
SPOOLing系统

输入 作业 作业控制进程

输出



典型同步/互斥机制

1. 信号量及 P、V 操作

1) 简介

- a) 一种经典的进程/线程同步机制，可以解决几乎所有同步问题
- b) 一种卓有成效的进程同步机制，由荷兰学者 Dijkstra 提出，1965 年

2) 信号量

- a) 一个特殊变量
- b) 用于进程线程间传递信号的一个整数值
- c) 最初提出的是二元信号量（互斥），之后推广到一般信号量（多值）或计数信号量（同步）

定义如下：

```
struc semaphore
{
    int count;
    queueType queue;
}
```

信号量说明：semaphore **s**;

d) 对信号量可以实施的操作

- 初始化（一般赋予信号量一些实际含义）
- P 和 V(P、V 分别是荷兰语的 test(proberen)和 increment(verhogen))

3) P、V 操作

允许 s.count < 0

P、V 操作定义

s 值恒大于等于 0

与一些参考书上或 ICS 课上定义不同，但效果是否一样？

允许 s.count < 0

```
P(s) test
{
    s.count --;
    if (s.count < 0)
    {
        该进程线程状态置为阻塞状态;
        将该进程插入相应的等待队列 s.queue 末尾;
        重新调度;
    }
}
```

不是忙等

调度时机

down, semWait

计数等待进程/线程

```
V(s)
{
    s.count ++;
    if (s.count <= 0)
    {
        唤醒相应等待队列 s.queue 中等待的一个进程线程;
        改变其状态为就绪态，并将其插入就绪队列;
    }
}
```

up, semSignal

看课件上 ics 对信号量的定义

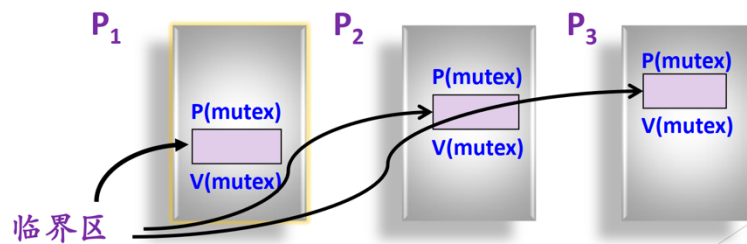
a) P、V 操作为原语操作

- 原语(primitive or atomic action)
- 完成某种特定功能的一段程序，具有不可分割性或不可中断性
- 原语的执行必须是连续的，在执行过程中不允许被中断
- 可以通过屏蔽中断、测试与设置指令等来实现

4) 用信号量解决进程线程间互斥问题

a) 思路

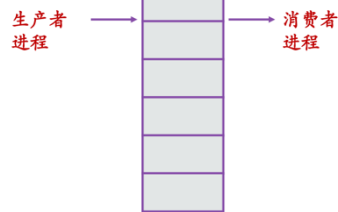
- 分析并发进程的关键活动，划定临界区
- 设置信号量 mutex，初值为 1
- 在临界区前实施 P(mutex)
- 在临界区之后实施 V(mutex)



5) 用信号量解决生产者/消费者(有界缓冲区)问题

问题描述:

- 一个或多个生产者生产某种类型的数据放置在缓冲区中
- 有消费者从缓冲区中取数据，每次取一项
- 只能有一个生产者或消费者对缓冲区进行操作



要解决的问题:

- 当缓冲区已满时，生产者不会继续向其中添加数据;
- 当缓冲区为空时，消费者不会从中移走数据

```

void producer(void)
{
    int item;
    while(TRUE) {
        item=produce_item();
        P(&empty);
        P(&mutex);
        insert_item(item);
        V(&mutex);
        V(&full);
    }
}

void consumer(void)
{
    int item;
    while(TRUE) {
        P(&full);
        P(&mutex);
        item=remove_item();
        V(&mutex);
        V(&empty);
        consume_item(item);
    }
}

#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
    
```

```

void producer(void)
{
    int item;
    while(TRUE) {
        item=produce_item();
        P(&empty);
        P(&mutex);
        insert_item(item);
        V(&mutex);
        V(&full);
    }
}

void consumer(void)
{
    int item;
    while(TRUE) {
        P(&full);
        P(&mutex);
        item=remove_item();
        V(&mutex);
        V(&empty);
        consume_item(item);
    }
}
    
```

若颠倒两个P操作的顺序? ☒ 错误!

若颠倒两个V操作的顺序? ☐ 正确!

先开读锁, 读的范围应尽可能小 (临界区)

6) 用信号量解决读者-写者问题

a) 问题介绍

- 问题描述
 - ✓ 多个进程共享一个数据区，这些进程分为两组：
 - ✓ 读者进程：只读数据区中的数据
 - ✓ 写者进程：只往数据区写数据
- 要求满足条件
 - ✓ 允许多个读者同时执行读操作（同步）
 - ✓ 不允许多个写者同时操作（互斥）
 - ✓ 不允许读者、写者同时操作（互斥）
- 应用场景
 - ✓ 买票
 - ✓ 路由器

b) 第一类：读者优先（写者要等读者读完）

- 如果读者执行
 - ✓ 若无其他读者、写者，该读者可以读
 - ✓ 若已有写者等，但有其他读者正在读，则该读者也可以读
 - ✓ 若有写者正在写，该读者必须等
- 如果写者执行
 - ✓ 若无其他读者、写者，该写者可以写
 - ✓ 若有读者正在读，该写者等待
 - ✓ 若有其他写者正在写，该写者等待

第一类读者-写者问题的解法

```
void reader(void)
{
    while (TRUE) {
        .....
        P(w);
        读操作
        V(w);
        .....
    }
}

void writer(void)
{
    while (TRUE) {
        .....
        P(w);
        写操作
        V(w);
        .....
    }
}
```

41

第一类读者-写者问题的解法

```
void reader(void)
{
    while (TRUE) {
        P(mutex);
        rc = rc + 1;
        if (rc == 1) P(w);
        V(mutex);
        读操作
        P(mutex);
        rc = rc - 1;
        if (rc == 0) V(w);
        V(mutex);
        其他操作
    }
}
```

42

```
void writer(void)
{
    while (TRUE) {
        .....
        P(w);
        写操作
        V(w);
    }
}
```

Linux: read_lock, write_lock

不需要 mutex
一次最多只能有一个写

2. 管程 (monitor)

1) 简介

- a) 是一种和特定语言关联的语言机制
- b) 主要通过条件变量以及 wait 和 signal 来实现
- c) The monitor is a programming-language construct that provides equivalent functionality to that of semaphores and that is easier to control.

2) 引入管程的原因

a) 信号量机制的不足

- 程序编写困难
- 效率低

b) 解决方案

Hoare (1974) & Brinch Hansen(1973)

- 在程序设计语言中支持管程成分
- 一种高级同步机制

3) 管程的定义

a) 简介

- 是一个特殊的模块
- 有一个名字
- 由关于共享资源的数据结构及在其上操作的一组过程组成

b) 进程与管程

- 进程只能通过调用管程中的过程来间接访问管程中的数据结构

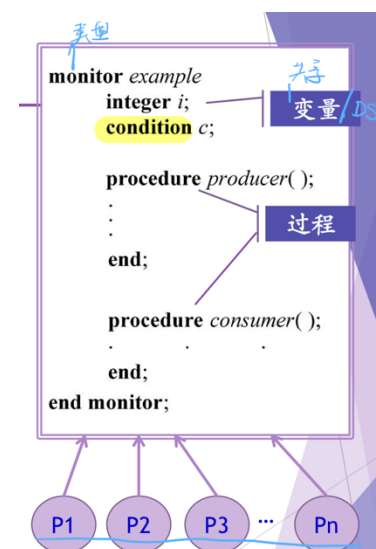
4) 管程应有的性质

a) 互斥进入

- 管程是互斥进入的，任何时刻只能有一个进程访问
- 为了保证管程中数据结构的数据完整性
- 注意：管程的互斥性是由编译器负责、保证的（编译器自动加解锁）

b) 支持进程解决同步问题

- 管程中设置条件变量（没有值，有队列，为进程提供等待的位置）



及等待/唤醒操作以解决进程的同步问题

- 可以让一个进程或线程在条件变量上等待（此时，应先释放该进程对管程的使用权，这样才能让其他进程进入管程来释放等待的进程），也可以通过发送信号将等待在条件变量上的进程或线程唤醒

5) 潜在问题

- a) 是否会出现这样一种情况，有多个进程同时在管程中？

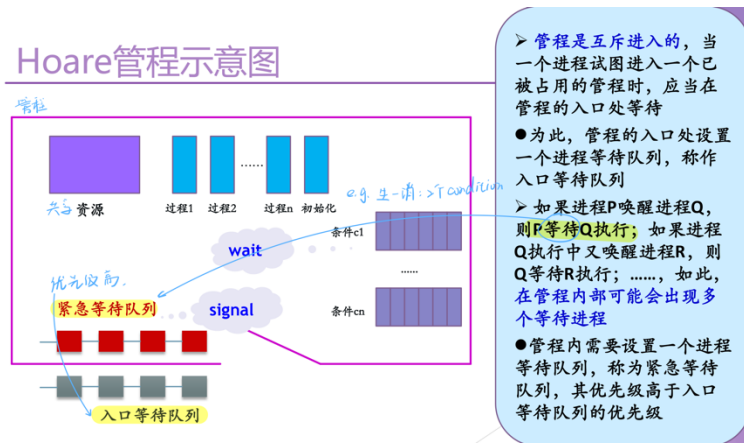
会出现，因此应当保证管程中任意时刻只有一个进程处于活跃状态

场景:	如何解决?
当一个进入管程的进程执行等待操作时，它应当释放管程的互斥权	处理方法:
当后面进入管程的进程执行唤醒操作时（例如，P唤醒Q），管程中便存在两个同时处于活动状态的进程	☐ P等待Q执行 (Hoare) V
	☐ Q等待P继续执行
	☐ 规定唤醒为管程中最后一个可执行的操作 (Hansen, 并发pascal)
	☐ 由系统决定 MESA

6) Hoare 管程

- a) 条件变量，wait 和 signal

- 条件变量 c



- ✓ 在管程内部说明和使用的一种特殊类型的变量
- ✓ `var c:condition;`
- ✓ 对于条件型变量，可以执行 wait 和 signal 操作
- wait(c)
 - ✓ 如果紧急等待队列非空，则唤醒第一个等待者，否则释放管

程的互斥权

✓ 执行此操作的进程进入 c 链尾部

• signal(c)

✓ 如果 c 链为空，则相当于空操作，执行此操作的进程继续执行；否则唤醒

✓ 第一个等待者，执行此操作的进程进入紧急等待队列的尾部

b) 用 Hoare 管程解决生产者消费者问题

```
monitor ProducerConsumer
condition full, empty;
integer count;
procedure insert (item: integer);
begin
    if count == N then wait(full);
    insert_item(item); count++;
    if count == 1 then signal(empty);
end;
function remove: integer;
begin
    if count == 0 then wait(empty);
    remove = remove_item; count--;
    if count == N-1 then signal(full);
end;
count:=0;
end monitor;
```

无返回
现在不在
buf 满
buf 空
唤醒消费者
唤醒生产者

```
procedure producer;
begin
    while true do
    begin
        item = produce_item;
        ProducerConsumer.insert(item);
    end
end;

procedure consumer;
begin
    while true do
    begin
        item=ProducerConsumer.remove;
        consume_item(item);
    end
end;
```

7) MESA 管程

a) 为什么有不同的管程？

解决问题场景：有多个进程同时在管程中

▶ 如何解决避免管程中同时有两个活跃进程？

确定 signal 之后的规则

▶ 三种方案

▶ Hoare 管程 后寻前

▶ Concurrent Pascal (Hansen) 最后再唤醒

▶ Mesa 管程

- 在管程设计时就确定了谁将有优先权，不灵活
- 发挥操作系统的作用

b) Hoare 管程的潜在问题与解决

• Hoare 管程中 signal 的缺陷

✓ 两次额外的进程切换 前上后下

✓ 是否会使条件队列中的进程永久挂起？难以证明

• 解决

✓ signal → notify

- ✓ notify: 当一个正在管程中的进程执行 notify(x) 时, 它使得 x 条件队列得到通知, 发信号的进程继续执行

c) notify

- notify 的结果
位于条件队列头的进程在将来合适的时候且当处理器可用时恢复执行
- 需要注意的问题
 - ✓ 由于不能保证在它 (再次上 CPU 运行) 之前没有其他进程进入管程, 因而这个进程必须重新检查条件
 - ✓ 因此应当用 while 循环取代 if 语句
 - ✓ 导致对条件变量至少多一次额外的检测 (但不再有额外的进程切换), 并且对等待进程在 notify 之后何时运行没有任何限制
- 改进
 - ✓ 给每个条件原语关联一个监视计时器, 不论条件是否被通知, 一个等待时间超时的进程将被设置为就绪状态
 - ✓ 当该进程被调度执行时, 会再次检查相关条件, 如果条件满足则继续执行
 - ✓ 可以防止如下情况的发生: 当某些进程在产生相关条件的信号之前失败时, 等待该条件的进程被无限制地推迟执行而处于饥饿状态

d) Broadcast 原语的引入

- broadcast 可以使所有在该条件上等待的进程都进入就绪状态
- 当一个进程不知道有多少进程将被激活时, 这种方式是很方便的
 - ✓ 例如, 在生产者/消费者问题中, 假设 insert 和 remove 函数都适用于可变长度的字符块, 此时, 如果一个生产者往缓冲区中添加了一批字符, 它不需要知道每个正在等待的消费者准备消耗多少字符, 而仅仅产生一个 broadcast, 所有正在等待的进程都得到通知并再次尝试运行 (可以让消费者各取所

需)

✓ 当一个进程难以准确判定将激活哪个进程时，也可使用广播

e) 用 Mesa 管程解决生产者-消费者问题

```
void append (char x)
{
    while(count == N) cwait(notfull); /* buffer is full; avoid overflow */
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    cnotify(notempty); /* one more item in buffer */
}

void take (char x)
{
    while(count == 0) cwait(notempty); /* buffer is empty; avoid underflow */
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    cnotify(notfull); /* one fewer item in buffer */
}
```

f) Hoare 管程与 Mesa 管程的比较

- Mesa 管程优于 Hoare 管程之处在于应用 Mesa 管程时出错比较少
- 在 Mesa 管程中，由于每个过程在收到信号后都检查管程变量，并且由于使用了 while 结构，一个进程不正确的广播或发信号，不会导致收到信号的程序出错
- 收到信号的程序将检查相关的变量，如果期望的条件没有满足，它会重新继续等待

8) 管程的实现

a) 直接构造

- 优点：效率高
- 构造方式
 - ✓ 定义一个抽象数据类型
 - ✓ 有一个明确定义的操作集合，通过它且只有通过它才能操纵该数据类型的实例
- 注意
 - ✓ 只能通过管程的某个过程才能访问资源；

- ✓ 过程是互斥的，某个时刻只能有一个进程或线程驻留在给定的管程中

Java中的管程(1/2)

```
public class ProducerConsumer {
    static final int N = 100; // constant giving the buffer size
    static producer p = new producer(); // instantiate a new producer thread
    static consumer c = new consumer(); // instantiate a new consumer thread
    static our_monitor mon = new our_monitor(); // instantiate a new monitor

    public static void main(String args[]) {
        p.start(); // start the producer thread
        c.start(); // start the consumer thread
    }

    static class producer extends Thread {
        public void run() { // run method contains the thread code
            int item;
            while (true) { // producer loop
                item = produce_item();
                mon.insert(item);
            }
        }
        private int produce_item() { ... } // actually produce
    }
}
```

Java中的管程(2/2)

MESA 语义

```
static class consumer extends Thread {
    public void run() { run method contains the thread code
        int item;
        while (true) { // consumer loop
            item = mon.remove();
            consume_item(item);
        }
    }
    private void consume_item(int item) { ... } // actually consume
}

static class our_monitor { // this is a monitor
    private int buffer[] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices
    public synchronized void insert(int val) {
        if (count == N) go_to_sleep(); // if the buffer is full, go to sleep
        buffer[hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N; // slot to place next item in
        count = count + 1; // one more item in the buffer now
        if (count == 1) notify(); // if consumer was sleeping, wake it up
    }
}
```

b) 间接构造 → 用某种已经实现的同步机制去构造

- 例如：用信号量及 P、V 操作构造管程

间接构造管程

首先，定义一个基础管程

Pascal

```
TYPE one_instance=RECORD
    mutex: semaphore; (初值1) 互斥用
    urgent: semaphore; (初值0)
    urgent_count: integer; (初值0)
END;

TYPE monitor_one =MODULE;
define enter,leave,wait,signal;

    mutex (入口互斥队列)
    urgent (紧急等待队列)
    urgent_count (紧急等待队列计数)
```

```
PROCEDURE enter(VAR
instance:one_instance);
BEGIN
    P(instance.mutex)
END;
```

```
PROCEDURE leave(VAR
instance:one_instance);
BEGIN
    IF instance.urgent_count > 0
    THEN
        BEGIN instance.urgent_count--;
            V(instance.urgent)
        END
    ELSE
        V(instance.mutex)
    END;
```

start_w 和 finish_w的实现

```
PROCEDURE start_w;
BEGIN
    monitor_one.enter(instance);
    write_count++;
    IF (write_count>1)
    OR(reading_count>0)
    THEN monitor_one.wait
        (instance,wq,w_count);
    monitor_one.leave(instance);
END;
```

```
V(instance.urgent);
END
ELSE
    V(instance.mutex);
P(s); 在s上等待
END;
```

```
PROCEDURE finish_w;
BEGIN
    monitor_one.enter(instance);
    write_count--;
    IF (r_count=0) THEN
        monitor_one.signal(instance,wq,w_count);
    ELSE
        monitor_one.signal(instance,rq,r_count);
    monitor_one.leave(instance);
END
```

```
V(s); 唤醒s上等待的
P(instance.urgent)
END
END;
```

用构造的管程解决读者-写者问题

```
TYPE r_and_w=MODULE;
VAR
    instance: one_instance;
    rq, wq: semaphore; (condition)
    r_count, w_count: integer;
    reading_count, write_count:
    integer;

define
    start_r, finish_r, start_w,
    finish_w;

use monitor_one.enter,
    monitor_one.leave, 基础类
    monitor_one.wait,
    monitor_one.signal;
```

```
BEGIN
    reading_count:=0;
    write_count:=0;
    r_count:=0;
    w_count:=0;
END;
```

读者的活动:

```
r_and_w.start_r;
读操作; 出来了则说明能读
r_and_w.finish_r;
```

写者的活动:

```
r_and_w.start_w;
写操作;
r_and_w.finish_w;
```

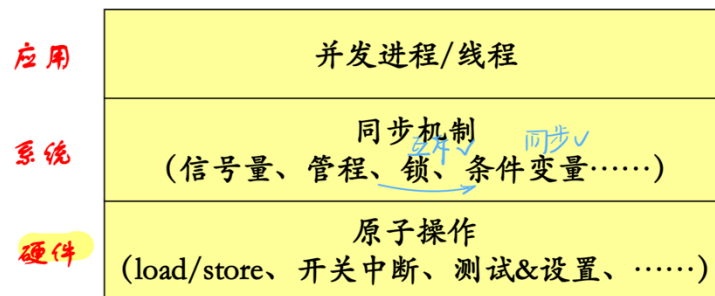
start_r 和 finish_r的实现

```
PROCEDURE start_r;
BEGIN
    monitor_one.Penter(instance);
    IF write_count>0 THEN
        monitor_one.wait 有写者在写
        (instance,rq,r_count);
    reading_count++;
    monitor_one.signal 可能有其他读者也在写 这个读者
        (instance,rq,r_count);
    monitor_one.leave(instance);
END;
```

```
PROCEDURE finish_r;
BEGIN
    monitor_one.enter(instance);
    reading_count--;
    IF reading_count=0 THEN
        monitor_one.signal(instance,wq,w_count);
    monitor_one.leave(instance);
END;
```

9) 总结：各种机制的层次

初表：用于内核开发



信号量一般只在内核中实现，因为保证原语操作需要开关中断

3. 锁与 Pthread

1) 锁（互斥量 mutex）

a) 简介

- 一种容易实现且有效的机制
- 适用于管理共享资源或一小段代码
- 常用于实现用户空间线程包
- 锁（互斥量）处于两种状态：
 - ✓ 加锁或解锁
 - ✓ 常为“锁”为一个整型量 → 0 为解锁/1 为加锁
- 提供两个过程
 - ✓ mutex_lock 和 mutex_unlock
 - ✓ 类似地，lock()和 unlock() / acquire()和 release()

```
acquire(lock)
...
critical section
...
release(lock)
```

b) 实现

1. 定义互斥锁类型，包括一个表示锁状态的布尔变量 (locked) 和一个等待锁的线程队列 (queue)。

```
struct mutex {
    bool locked;
    queue_t queue;
};
```

2. 初始化锁为未锁定状态。

```
void mutex_init(mutex_t *mutex) {
    mutex->locked = false;
    init_queue(&mutex->queue);
}
```

3. 加锁操作，当锁已经被其他线程持有时，当前线程将阻塞并加入到等待队列中。

```
void mutex_lock(mutex_t *mutex) {
    while (atomic_swap(&mutex->locked, true) == true) {
        // 等待锁释放
        add_to_queue(&mutex->queue, current_thread);
        yield_cpu(); // 让出CPU时间片
    }
}
```

4. 解锁操作，当锁被释放后，如果有等待线程则唤醒其中一个线程继续执行。

```
void mutex_unlock(mutex_t *mutex) {
    atomic_swap(&mutex->locked, false);
    if (!is_queue_empty(&mutex->queue)) {
        // 唤醒等待的线程
        thread_t *next_thread = remove_from_queue(&mutex->queue);
        wakeup_thread(next_thread);
    }
}
```

锁的实现伪码—ChatGPT

- 有问题的方法 1:通过开关中断指令实现

```
lock() {
    disable interrupts
    while (value != FREE);
    value = BUSY
    enable interrupts
}
```

• 等待锁变为空闲
• 获得锁

只针对当前CPU.

思考: 这样实现锁是否正确?

单CPU X 死循环
多CPU X 中断被屏蔽

- ✓ 解决方案

```
lock() {
    disable interrupts
    while (value != FREE) {
        enable interrupts
        disable interrupts
    }
    value = BUSY
    enable interrupts
}
```

设计一个方案
编码验证一下

- 有问题的方法 2

另一种方案

无忙等待 (睡眠)
+
开关中断

```
lock() {
    disable interrupts
    if (value == FREE) {
        value = BUSY
    } else {
        add thread to queue of threads waiting for this lock
        switch to next run-able thread
    }
    enable interrupts
}
```

需要大家思考的问题: 在①或②或③处开中断是否可行? 为什么?

问题: 还是关中断的状态

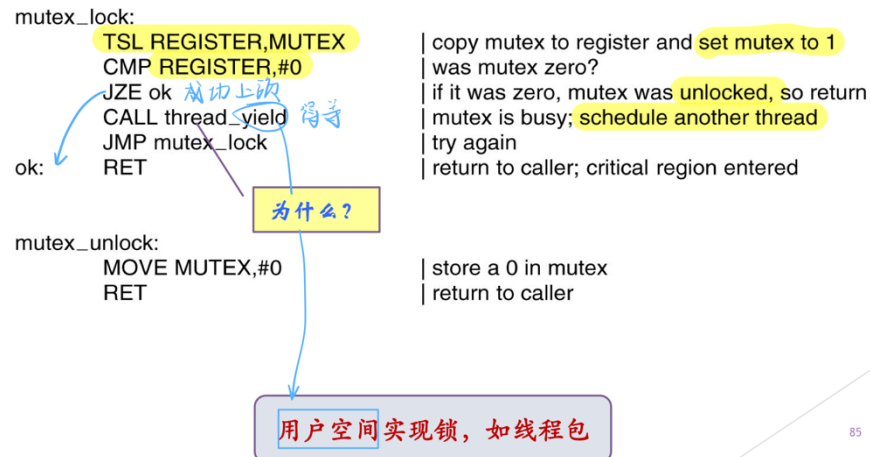
是否 or 就响? 二进制

- ✓ 解决方案

```
Thread A
enable interrupts
}
<user code runs>
lock() {
    disable interrupts
    ... 前边: A 拿不到锁
    switch
    ...
    back from switch
    enable interrupts
    }
    <user code runs>
    unlock() (move thread A to ready queue)
    yield() {
        disable interrupts
        switch
    }
}

Thread B 持有锁
yield() {
    disable interrupts
    switch
    ...
    back from switch
    enable interrupts
    }
    <user code runs>
    unlock() (move thread A to ready queue)
    yield() {
        disable interrupts
        switch
    }
}
```


- 通过 Test&Set 指令实现锁



2) Pthreads 中的同步互斥机制

yield 让出CPU 用户空间 runtime system 管理线程

Pthreads中的同步机制

Thread call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy an existing mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acquire a lock or fail
Pthread_mutex_unlock	Release a lock

互斥量
Pthreads中保护临界区

条件变量
Pthreads中的
同步机制

Thread call	Description
Pthread_cond_init	Create a condition variable
Pthread_cond_destroy	Destroy a condition variable
Pthread_cond_wait	Block waiting for a signal
Pthread_cond_signal	Signal another thread and wake it up
Pthread_cond_broadcast	Signal multiple threads and wake all of them

3) 用 Pthreads 解决生产者-消费者问题

```

int main(int argc, char **argv)
{
    pthread_t pro, con;
    pthread_mutex_init(&the_mutex, 0);
    pthread_cond_init(&condc, 0);
    pthread_cond_init(&condp, 0);
    pthread_create(&con, 0, consumer, 0);
    pthread_create(&pro, 0, producer, 0);
    pthread_join(pro, 0);
    pthread_join(con, 0);
    pthread_cond_destroy(&condc);
    pthread_cond_destroy(&condp);
    pthread_mutex_destroy(&the_mutex);
}

```

- ① X
- ② 信号量
- ③ 线程 > 种
- ④ Pthreads

```

#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000 /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0; /* buffer used between producer and consumer */

void *producer(void *ptr) /* produce data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr) /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

```

① P(同步信号量) ② P(互斥锁) ③ P(互斥锁)

MESA

两个同步问题

一个缓冲区

75

Pthreads中的同步机制

```

#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000 /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0; /* buffer used between producer and consumer */

void *producer(void *ptr) /* produce data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        if (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr) /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        if (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

```

Hoare

一个缓冲区

78

Pthreads中的同步机制

a) pthread_cond_wait

- 当发起一个 pthread_cond_wait 之后，分解为三个动作
 - ✓ 解锁
 - ✓ 等待
 - ✓ 当收到一个解除等待的信号 (pthread_cond_signal 或者 pthread_cond_broadcast) 之后，pthread_cond_wait 马上要做的动作是：上锁 (避免死锁)

Linux 内核同步机制

原子操作

自旋锁

读写自旋锁

信号量

读写信号量

互斥体

完成变量

顺序锁

屏障

...

1. 原子操作

1) 简介

- a) 不可分割，在执行完之前不会被其他任务或事件中断
- b) 常用于实现资源的引用计数
- c) `atomic_t`

```
typedef struct { volatile int counter; } atomic_t;
```

原子操作API包括:

```
atomic_read(atomic_t *v);  
atomic_set(atomic_t *v, int i);  
void atomic_add(int i, atomic_t *v);  
int atomic_sub_and_test(int i, atomic_t *v);  
void atomic_inc(atomic_t *v);  
int atomic_add_return(int i, atomic_t *v);
```

► 例子

```
atomic_t v = atomic_init(0);
```

```
atomic_set(&v, 4);  
atomic_add(2, &v);  
atomic_inc(&v);  
printk("%d\n", atomic_read(&v));
```

```
int atomic_dec_and_test(atomic_t *v)
```

```
原子操作演示  
atomic_t my_counter ATOMIC_INIT(0);  
atomic_add( 1, &my_counter );  
atomic_inc( &my_counter );  
atomic_sub( 1, &my_counter );  
atomic_dec( &my_counter );
```

```
1 include/linux/types.h  
typedef struct {  
    int counter;  
} atomic_t;
```

```
2 设置 加减等实现与具体的arch相关 arch/x86/include/asm/atomic.h  
static inline void atomic_add(int i, atomic_t *v)  
{  
    asm volatile(LOCK_PREFIX "addl %1,%0"  
        : "+m" (v->counter)  
        : "ir" (i));  
}
```

2. 自旋锁

1) 简介

- a) 为防止多处理器并发而引入的一种锁
- b) 在内核中大量应用于中断处理等部分

- 问题：在单 CPU 上如何解决中断处理过程中的并发？

- c) 自旋锁最多只能由一个内核任务持有，如果一个内核任务试图请求一个已被占用的自旋锁，这个任务会一直忙等待（旋转）等待锁重新可用
- d) 可以在任何时刻防止多于一个的内核任务同时进入临界区，因此这种锁可有效地避免多处理器上并发运行的内核任务竞争共享资源

2) 例子

```
#include <linux/spinlock.h> //自旋锁声明
spinlock_t mylock = SPIN_LOCK_UNLOCKED; /* 初始化 */

/* 获取自旋锁 */
spin_lock(&mylock);

/* ... 临界区代码 ... */

spin_unlock(&mylock); /* 释放锁 */
```

3) 实现

传统自旋锁的实现

spinlock_t 类型定义

```
typedef struct arch_spinlock {
    unsigned int slock;
} arch_spinlock_t;
```

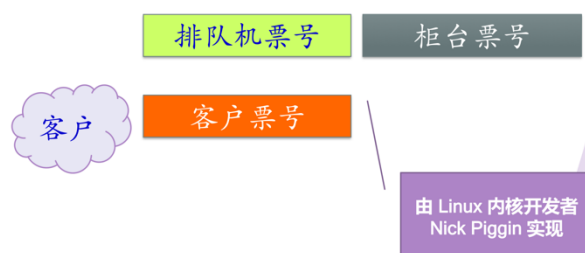
```
static inline void
__raw_spin_unlock(raw_spinlock_t *lock)
{
    asm volatile("movb $1,%0" : "+m" (lock->slock) :: "memory");
}
```

```
static inline void
__raw_spin_lock(raw_spinlock_t *lock)
{
    asm volatile("\n1:\t"
        LOCK_PREFIX " ; decb %0\n\t"
        "jns 3f\n\t"
        "2:\t"
        "rep;nop\n\t"
        "cmpb $0,%0\n\t"
        "jle 2b\n\t"
        "jmp 1b\n\t"
        "3:\n\t"
        : "+m" (lock->slock) : : "memory");
}
```

这样实现有什么问题？

4) 排队自旋锁

类似银行营业厅排队票号系统



排队机票号 柜台票号

```

typedef struct arch_spinlock {
    unsigned int slock;
} arch_spinlock_t;

static inline void __raw_spin_lock(raw_spinlock_t *lock)
{
    short inc = 0x0100; //客户票据 高8位表示自己票号,
                        // 低8位表示查看到的柜台票号, 低8位在忙等待过程中会持续更新
    __asm__ __volatile__(
        LOCK_PREFIX "xaddw %w0, %1\n" //客户获得票据, 以及当前柜台的票号;
                                //排队机票号加1
        "1:\n"
        "cmpb %h0, %b0\n\t" //客户比较自己票号与柜台票号
        "je 2f\n\t" //如果相等, 说明空闲可用, 就去办理业务
        "rep ; nop\n\t" //如果不相等, 则空等一个指令
        "movb %1, %b0\n\t" //获取柜台最新票号
        "jmp 1b\n\t" //跳转到标签为1的位置, 准备再一次检查能否获得锁
        "2:"
        : "+Q" (inc), "+m" (lock->slock)
        :
        : "memory", "cc");
}

static inline void __raw_spin_unlock(raw_spinlock_t *lock)
{
    __asm__ __volatile__(
        //办理完业务后 将柜台票号加1
        UNLOCK_LOCK_PREFIX "incb %0"
        : "+m" (lock->slock)
        :
        : "memory", "cc");
}

```

3. 信号量

1) 定义与操作

semaphore 类型定义

```

struct semaphore {
    spinlock_t lock; |
    unsigned int count;
    struct list_head wait_list;
};

```

semaphore 主要操作

```

extern void down(struct semaphore *sem);
extern int __must_check down_interruptible(struct semaphore *sem);
extern int __must_check down_killable(struct semaphore *sem);
extern int __must_check down_trylock(struct semaphore *sem);
extern int __must_check down_timeout(struct semaphore *sem, long jiffies);
extern void up(struct semaphore *sem);

```

2) 自旋锁与信号量

- ▶ 单处理器？与多处理器
- ▶ 睡眠锁 与 忙等锁
- ▶ 应用场合分析

思考题

应用场合	选择（选一或选二或优选）
临界区执行时间较快	
临界区执行时间较长	
临界区可能包含引起睡眠的代码	
.....	

4. 读写锁

1) 应用场景

如果每个执行实体对临界区的访问或者是读或者是写共享数据结构，但是它们都不会同时进行读和写操作，读写锁是最好的选择

2) 实例：Linux 的 IPX 路由代码使用了读-写锁，用 `ipx_routes_lock` 的读-写锁保护 IPX 路由表的并发访问

a) 要通过查找路由表实现包转发的执行单元需要请求读锁；需要添加和删除路由表中入口的执行单元必须获取写锁（由于通过读路由表的情况比更新路由表的情况多得多，使用读-写锁提高了性能）

3) 使用读写锁的例子

```
rwlock_t myrwlock = RW_LOCK_UNLOCKED;

write_lock(&myrwlock); /* 获取写锁 */
/* ... 临界区代码 ... */
write_unlock(&myrwlock); /* 释放写锁 */
```

```
rwlock_t myrwlock = RW_LOCK_UNLOCKED;

read_lock(&myrwlock); /* 获取读锁 */
/* ... 临界区代码 ... */
read_unlock(&myrwlock); /* 释放读锁 */
```

4) RCU 机制(Read-Copy Update)

a) Linux 2.6 之后，一种数据一致性访问机制

b) 对于读操作

- 直接对共享资源进行访问（需要 CPU 支持访存操作的原子化）
- 采用 `read_rcu_lock()`，RCU 的读操作上下文是不可抢占的

c) 对于写操作

- 将原来的老数据作一次备份(copy)，然后对备份数据进行修改，修改完毕之后再用新数据更新老数据，更新老数据时采用了 `rcu_assign_pointer()` 宏，之后，需要进行老数据资源的回收
- 采用数据备份的方法可以实现读者与写者之间的并发操作，但是不能解决多个写者之间的同步，所以当存在多个写者时，需要通过锁机制对其进行互斥，也就是在同一时刻只能存在一个写者

d) 读操作几乎没有开销，写操作开销比较大，适用于读操作很多、写操作极少的场景

5. 完成变量 (Completion variable)

1) 简介

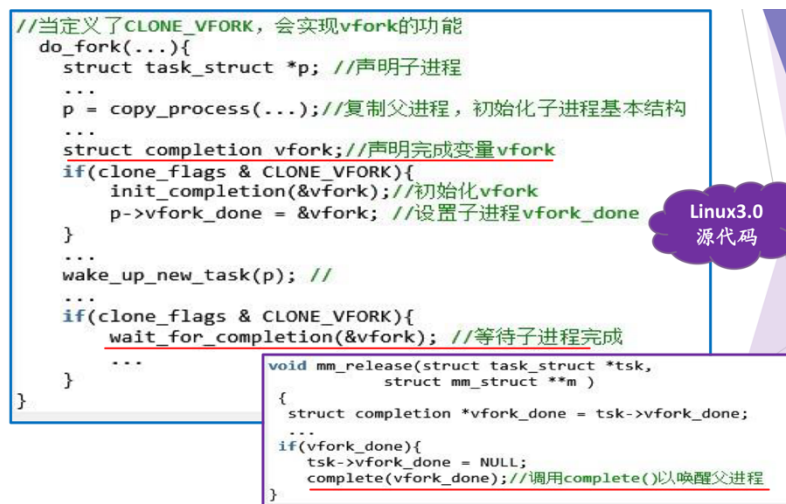
a) 如果内核中一个进程需要发出信号通知另一进程发生了某特定事件，

利用完成变量是使两进程得以同步的简单方法

b) 与信号量思想类似

方法	描述
<code>init_completion(struct completion *)</code>	初始化指定的动态创建的完成变量
<code>wait_for_completion(struct completion *)</code>	等待指定的完成变量接收信号
<code>complete(struct completion *)</code>	发信号唤醒等待进程

2) 应用



6. 顺序锁

1) 简介

a) 顺序锁用于读写共享数据

- 这种锁依靠一个序列计数器
- 写时：会得到一个锁，序列值增加
- 读时：读前读后都读取序列值，若相同，则成功读

b) 使用场景

- 读者很多，写者很少
- 虽然写者少，但希望写优先于读，而且不允许读者让写者饥饿
- 操作的数据很简单

2) 应用

jiffies_64用来存储Linux机器启动到当前的总时间

```
//读取jiffies_64
unsigned long long get_jiffies_64(void)
{
    unsigned long seq;
    unsigned long long ret;
    do {
        seq = read_seqbegin(&xtime_lock); //准备读, 返回顺序锁的顺序号
        ret = jiffies_64; //读取数据
    } while (read_seqretry(&xtime_lock, seq)); //检查读的过程中有没有写者访问了共享资源
        //如果有 则进入循环重新读
        //如果成功完成读操作
    return ret; }

//修改jiffies_64
write_seqlock(&xtime_lock); //加顺序锁
++jiffies_64; //修改数据
write_sequnlock(&xtime_lock); //解锁
```

7. 屏障(Barrier)

- 另一种同步机制(又称栅栏、关卡)
- 用于对一组线程进行协调
- 应用场景

一组线程协同完成一项任务, 需要所有线程都到达一个汇合点后再一起向前推进

思考题:
如何用P、
V操作实
现?

